

universidad de León Online

Universidad de León ▪ Máster Universitario en Ciberseguridad

Ingeniería del software seguro en la era de la IA generativa

Diseño y Programación Seguras

Caso de estudio transversal

Francisco Javier Rodríguez Lera

Material elaborado con asistencia de IA generativa (DeepSeek-V4-Flash; considerados también GPT-6 y Claude Sonnet 4.6). Contenido revisado y validado por el autor.



Instituto de Investigación e Innovación en Ingeniería

Escuela de Ingenierías Industrial, Informática y Aeroespacial

universidad de León

Objetivos y naturaleza del caso

- Presentar **un mismo sistema** a lo largo de todo el libro.
- Dar un escenario concreto a los conceptos de especificación, verificación y riesgo.
- Permitir comparar, sobre el mismo enunciado, código vulnerable, corregido y generado por IA.

Etiquetado epistémico. El conversor es un **ejemplo docente**: no es un sistema industrial real. No se presentan resultados medidos de rendimiento, disponibilidad ni seguridad. Los fragmentos de código son *ilustrativos y contextuales*; las partes sin código se marcan como **propuestas**.

Contexto y motivación

Procesar datos estructurados es habitual, pero la aparente sencillez es engañosa:

- Leer un CSV implica decidir comillas, separadores, fin de línea y codificación.
- Validar implica distinguir una entrada *malformada* de una *hostil*.
- Generar PDF/XLSX implica incorporar bibliotecas con sus propias superficies de ataque.
- Escribir ficheros implica decidir dónde y con qué permisos.

Ninguna de esas decisiones es neutral para la seguridad.

Por qué este caso

El caso es apropiado por tres razones:

1. **Cabe en una práctica** y se entiende sin contexto de dominio.
2. Obliga a tratar a la vez **entrada no confiable**, **transformación de datos** y **salida a disco**: las tres fronteras donde se concentran las familias de fallo.
3. Permite comparar código vulnerable, corregido y generado por IA sobre el mismo enunciado, atribuyendo diferencias a decisiones de diseño.

Alcance funcional

Categoría	Funcionalidad	Estado
Confirmado	Leer CSV y separar columnas	Descrito
Confirmado	Validar la entrada antes de procesar	Descrito
Confirmado	Normalizar columnas y codificación	Descrito
Confirmado	Generar un documento PDF	Descrito
Confirmado	Exportar a XLSX	Descrito
Confirmado	Gestionar errores de entrada y escritura	Descrito
Propuesto	Configuración explícita (límites, rutas)	Enunciado; sin código
Propuesto	Registro de operaciones y observabilidad	Enunciado; sin código
Fuera de alcance	Edición colaborativa, servicio en red, GUI	No forma parte
Fuera de alcance	Conversión inversa (PDF/XLSX $\rightarrow$ CSV)	No reversible

El alcance descrito es el *modelo* del sistema, no un producto terminado.

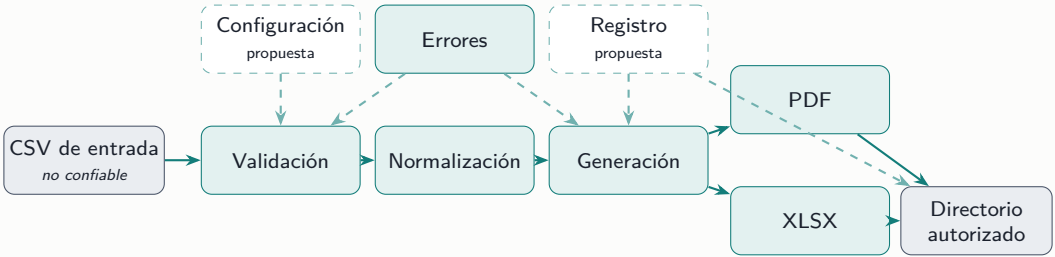
Requisitos no funcionales

Se enuncian como *criterios verificables*, no como valores medidos.

- **Seguridad**: no escribir fuera del directorio autorizado; sin celdas interpretables como fórmulas.
- **Robustez**: entrada malformada rechazada con error explícito.
- **Rendimiento**: tiempo y memoria acotados ante tamaño máximo.
- **Mantenibilidad**: decisiones explícitas; complejidad limitada.
- **Portabilidad**: comportamiento declarado por plataforma.
- **Trazabilidad**: cada requisito ligado a comprobación y evidencia.
- **Testabilidad**: propiedades críticas expresables como pruebas.
- **Recuperabilidad**: restaurar el estado anterior tras un fallo.
- **Reversibilidad**: revertir código, despliegue y estado.

La distinción *requisito* vs. *resultado* es deliberada: un requisito de seguridad no dice que el sistema sea seguro.

Arquitectura de referencia



Dos rasgos fijos: (i) *frontera de confianza* -el CSV se trata como hostil; (ii) *confinamiento de salida* -escritura solo en el directorio autorizado. Trazo discontinuo: componentes *propuestos*, no implementados.

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

Frontera de confianza y confinamiento

Frontera de confianza

- El CSV de entrada es *dato no confiable*.
- Toda la cadena posterior asume que puede ser hostil.
- Se repite en vulnerabilidades, portabilidad, verificación y validación.

Confinamiento de salida

- La escritura se limita al directorio autorizado.
- No se siguen enlaces que permitan escapar de él.
- Conecta con aislamiento técnico y validación.

Modelo de datos

El caso maneja tres representaciones; la conversión no conserva toda la información:

Entrada (CSV). Texto tabular con separador, comillas y fin de línea explícitos; codificación declarada (UTF-8); valores tratados como cadenas hasta validarlos.

Representación intermedia. Filas y columnas validadas y normalizadas; los tipos los decide la especificación.

Salida (PDF y XLSX). Salidas *terminales*: el PDF presenta; el XLSX interpreta ciertos prefijos como fórmulas.

Un valor que en el CSV es una cadena puede convertirse, al exportar a XLSX, en una fórmula que la hoja evalúa. La conversión **no es reversible**.

Contratos e invariantes

Invariantes de seguridad especificadas (no resultados verificados sobre una implementación completa):

- **Contención de escritura**: ninguna operación escribe fuera del directorio autorizado.
- **Ausencia de fórmulas**: ninguna celda del XLSX contiene contenido interpretable como fórmula.
- **Acotación de recursos**: el proceso termina en tiempo y memoria acotados.

Contratos de entrada (desarrollados en el capítulo de correctitud): precondiciones (qué CSV se acepta), poscondiciones (qué debe cumplir la salida), invariantes (qué se mantiene durante el proceso).

¿Qué es una invariante verificable?

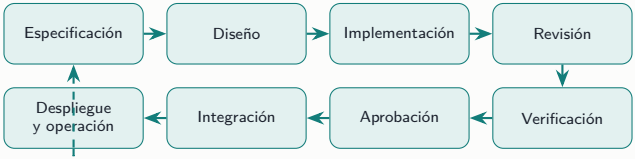
- Un enunciado vago («no confiar en el CSV») ¿es una invariante?
- ¿Qué hace falta para convertir una intención en una propiedad comprobable?

Modelo de amenazas del caso

- **Archivos no confiables**: CSV malformado o diseñado para consumos excesivos.
- **Contenido malformado**: comillas, separadores y codificaciones ambiguas.
- **Consumo excesivo**: tamaño máximo o ratio de compresión extremo.
- **Rutas y nombres**: *path traversal*, rutas absolutas, enlaces simbólicos.
- **Dependencias externas**: bibliotecas de PDF/XLSX, versiones y vulnerabilidades.
- **Fórmulas en hojas**: campos que empiezan por `=`, `+`, `-` o `@`.
- **Información sensible**: datos personales o secretos que la salida pudiera exponer.

El desarrollo del modelo de amenazas de las *acciones del agente* corresponde al capítulo de aislamiento técnico.

Ciclo de vida del caso



Es un *ciclo*, no una secuencia lineal: la evidencia de la operación realimenta la revisión de requisitos, y la reversibilidad permite volver a un estado anterior. No todas las fases están implementadas con código.

Reversibilidad del caso

Cinco operaciones que se confunden bajo el mismo término:

1. **Revertir una modificación del código**: deshacer un cambio en el fuente.
2. **Recuperar una versión anterior del sistema**.
3. **Restaurar archivos o datos modificados** (salidas anteriores).
4. **Compensar efectos externos** (correo enviado, publicación).
5. **Recuperar un estado operativo aceptable**.

Las salidas son terminales y la conversión no es reversible. Lo que debe ser reversible es el *proceso*: restaurar salidas anteriores y volver a una versión conocida del código.

Evolución del caso a lo largo del libro

Sección / capítulo	Principio	Elemento del caso
Introducción	Hilo conductor	Presentación del escenario
Principios	KISS, YAGNI, DRY	Alcance: cuántos formatos
Correctitud	Especificación	Contrato del conversor
Vulnerabilidades	Familias CWE	Cada punto de fallo
Portabilidad	Portabilidad	Rutas, temporales, codificación
Evaluación	Reproducibilidad	El caso como unidad de análisis
Confianza	Confianza no calibrada	Código plausiblemente incorrecto
Aislamiento	Agentes	Modelo de amenazas del flujo
Validación	Pipeline	Requisitos como entradas
Verificación	Contratos	Contención, fórmulas, acotación
Metodología	AI-SDLC	Aprobación, operación, reversibilidad

La evolución es de *perspectiva*, no de versión: cada sección añade una capa de análisis sobre el mismo modelo.

Desarrollo humano frente a asistido por IA

Mismos requisitos y criterios de aceptación; cambia *cómo* se obtiene y justifica cada artefacto:

- **Obtención de requisitos**: entrevista humana vs. relleno de huecos implícitos por el asistente.
- **Trazabilidad de decisiones**: el proceso humano suele dejar registro; el asistido puede no exponer el razonamiento.
- **Revisión**: debe ser independiente del origen del código.
- **Pruebas**: el asistente puede reflejar el código, no la especificación (circularidad).
- **Documentación**: puede ser plausible sin corresponder a la implementación real.

Las propiedades de seguridad dependen de las decisiones de diseño y de la verificación, no de quién escribió el código.

Relación con el ejemplo técnico de suma comprobada (C23)

El capítulo incluye además un ejemplo técnico acotado: la función de suma comprobada de enteros sin signo en C23 (`checked_add_u32`).

- **Finalidad distinta** del caso transversal: formalizar contratos, comprobar desbordamientos, aplicar pruebas de propiedades.
- Componente pequeño: las técnicas formales se presentan mejor sobre una función con contrato explícito que sobre el conversor completo.
- **No es un segundo caso transversal**: es un ejemplo técnico dentro de la verificación (capítulo 11; desarrollo completo en el Anexo B).

Limitaciones del caso

- Es un **ejemplo docente**, no un sistema industrial: sin mediciones de rendimiento, disponibilidad ni seguridad.
- Los fragmentos de código son *ilustrativos*; no son una implementación completa ni verificada.
- La arquitectura incluye componentes *propuestos* (configuración, registro) sin código.
- La conversión no es reversible: la reversibilidad se refiere al proceso.
- No hay una serie de versiones numeradas; la evolución es de perspectiva.

Del prompt al proceso de ingeniería: tres aproximaciones

Aproximación	Qué produce sobre el caso	Qué no garantiza
A. Prompt-driven	Código del conversor desde instrucciones en lenguaje natural	Requisitos enumerados, trazabilidad, cobertura, evidencias
B. Specification-driven	Requisitos (EARS), plan, tareas, implementación y pruebas	Que la especificación sea correcta o completa; garantías formales
C. Ciclo de ingeniería asistido por IA	B, más verificación con evidencias, aprobación, operación y reversibilidad	Que las pruebas sean exhaustivas; corrección general

No son excluyentes ni una escala de madurez universal. Generar código, estructurarlo y obtener garantías son actividades *distintas*.

Prompt-driven vs. Specification-driven

A. Prompt-driven

- «Lee un CSV y genera PDF y XLSX validando los datos».
- Produce un borrador plausible.
- *No* deja rastro de qué se pidió exactamente.
- Sin requisitos, criterios de aceptación ni pruebas ligadas.

B. Specification-driven

- Requisitos numerados (EARS), criterios y casos límite.
- De ellos derivan plan, tareas, implementación y pruebas.
- Hace *explícito* el vínculo requisito→prueba.
- *No* garantiza que la especificación sea correcta o completa.

La calidad de la especificación sigue siendo responsabilidad humana. Iniciativas como Hello SDD [2] y GitHub Spec Kit [1] documentan el flujo SDD.

Ciclo de ingeniería asistido por IA

La tercera aproximación amplía la SDD con los mecanismos del libro:

- **Verificación con evidencias** (capítulo 11).
- **Separación entre verificar y aprobar**.
- **Despliegue controlado**, observabilidad y recuperación.
- **Reversibilidad** como propiedad transversal.

Es la diferencia entre *estructurar* el desarrollo y *establecer garantías* de ingeniería: estas últimas no se automatizan por disponer de un agente o de un flujo de trabajo.

Síntesis del caso transversal

- Un mismo sistema (CSV–PDF–XLSX) recorre todo el libro como **hilo conductor**.
- **Frontera de confianza** y **confinamiento de salida**: dos rasgos fijos.
- Tres representaciones de datos; conversión **no reversible**.
- Invariantes de seguridad *especificadas*, no resultados verificados.
- El ciclo de vida es un **ciclo**, con reversibilidad del proceso.
- Tres aproximaciones (prompt / SDD / ciclo): generar, estructurar y garantizar son actividades distintas.

Actividad

1. Dibuje la frontera de confianza del conversor y señale qué componentes tratan datos no confiables.
2. Distinga, en el caso, un requisito *confirmado* de uno *propuesto*.
3. Formule una invariante vaga («validar bien la entrada») y conviértala en una propiedad observable.

¿Puede una conversión ser reversible?

- ¿Qué se pierde al convertir CSV en PDF o XLSX?
- ¿Qué significa, entonces, «reversibilidad» en este caso?

Referencias I



GitHub.

Spec kit: toolkit for specification-driven development (sdd).

Documentación técnica / propuesta metodológica. Consultado en noviembre de 2026.



Brais Moure.

Hello sdd: curso de spec-driven development (sdd) desde cero.

Documentación técnica / material docente. Consultado en noviembre de 2026.